

Driver Control Quality

Make TeleOp usable before adding complexity.

For a new FTC team, the first goal after "the robot drives" is not autonomous, vision, or advanced mechanisms. The first goal is that the drivers can control the robot confidently, repeatably, and safely for an entire match.

Good driver control is engineering work. It comes from clear controls, predictable defaults, safe mechanism behavior, and enough practice time for the drive team to build muscle memory.

Goals

A good TeleOp should be:

- Predictable: the same input always produces the same behavior.
- Easy to recover: releasing controls returns the robot to a safe state.
- Comfortable: common actions are easy to reach without awkward hand movement.
- Separated: the driver drives; the operator manages mechanisms.
- Observable: telemetry shows enough state to debug problems quickly.
- Safe: mechanisms stop or hold safely when controls are released.

Slow Mode / Precision Mode

Slow mode gives the driver fine control near field elements, game pieces, and scoring targets.

Without slow mode, a mecanum robot can be too twitchy for precise alignment. Drivers often need full speed for crossing the field and reduced speed for scoring.

Common implementation:

```
normal drive power = joystick value  
slow drive power = joystick value * 0.3 to 0.6
```

Recommended controls:

- Hold right bumper for slow mode.
- Keep rotation either unchanged or slightly reduced.
- Make slow mode active only while the button is held.

Example behavior:

```
if slow button is held:  
    x *= 0.5  
    y *= 0.5  
    turn *= 0.5
```

Testing checklist:

- Driver can approach a wall or target without overcorrecting.
- Robot still moves reliably at low battery.
- Slow mode does not reverse any direction.
- Releasing slow mode smoothly returns to full-speed control.

Robot-Centric vs Field-Centric Driving

Robot-Centric

Robot-centric driving means joystick directions are relative to the robot.

stick forward = robot drives toward its own front
stick right = robot strafes toward its own right

This is simplest for new teams and requires no heading sensor.

Pros:

- Easy to implement.
- Easy to debug motor directions.
- Works without IMU.
- Good first choice for a new team.

Cons:

- Driver must mentally track robot orientation.
- Driving is harder when the robot faces the driver.

Field-Centric

Field-centric driving means joystick directions are relative to the field.

stick forward = robot drives away from driver station
stick right = robot moves field-right

The code rotates the joystick vector by the robot's heading from the IMU.

Pros:

- Easier for many drivers after practice.
- Useful when the robot turns frequently.
- Reduces orientation confusion.

Cons:

- Requires reliable IMU heading.
- Needs heading reset/calibration.
- Bad heading data makes driving confusing.

Recommended progression:

```
start robot-centric  
-> add heading telemetry  
-> add field-centric as optional mode  
-> let drivers choose after practice
```

Do not force field-centric driving before the team understands robot-centric drive and IMU reset behavior.

Button Mapping Discipline

Button mapping should be treated like a user interface.

Bad button maps cause missed scores, broken mechanisms, and driver confusion. Good button maps make common actions natural and rare actions hard to trigger accidentally.

Rules:

- Put common actions on easy buttons.
- Put dangerous actions behind holds, combinations, or less convenient buttons.
- Avoid putting opposite actions next to each other unless they are clearly paired.
- Keep the same button meaning throughout the season.
- Do not change controls right before a match unless absolutely necessary.
- Document controls in the repo.

Good patterns:

- Hold button: temporary behavior like slow mode or manual override.
- Toggle button: persistent state like intake on/off, but only if state is obvious.
- D-pad: mode selection or mechanism presets.
- Triggers: proportional control, such as turret or arm jog.
- Bumpers: quick actions used while driving.

Avoid:

- Multiple unrelated actions on one button.
- Hidden state that is not shown in telemetry.
- A button that starts a mechanism but no obvious button to stop it.
- Controls where releasing the button leaves a motor running unexpectedly.

Controller Handoff: Driver vs Operator

Use two gamepads when the robot has both drivetrain and mechanisms.

Recommended split:

Driver, Gamepad 1

Owns robot movement:

- forward/back
- strafe
- turn
- slow mode
- heading reset if using field-centric
- driver-facing alignment assist

The driver should not need to operate intake, shooter, lift, or scoring mechanisms during normal play.

Operator, Gamepad 2

Owns mechanisms:

- intake
- outtake/reverse
- lift/arm/turret
- shooter
- gate/claw/servo actions
- scoring presets
- mechanism emergency stop

The operator should not control chassis movement except for carefully designed alignment assists.

Why this matters:

- The driver can focus on positioning.
- The operator can focus on game-piece flow and scoring.
- Fewer accidental inputs happen during stressful matches.
- Practice roles become clear.

Handoff rule:

Only one controller should own each subsystem during normal TeleOp.

If both controllers can command the same mechanism, define priority explicitly.

Safe Defaults

Every TeleOp loop should have safe behavior when no relevant button is pressed.

For drivetrain:

no joystick input -> all drive motor powers zero

For mechanisms:

no command -> motor stopped, held, or in a known safe state

Safe default examples:

- Intake stops when intake button is released, unless intentionally toggled.
- Arm holds position instead of falling.
- Shooter turns off when disabled or idle.
- Servo gate returns to closed unless scoring.
- Manual jog stops when trigger is released.

State machines should also have safe defaults:

unknown state -> stop mechanism and report error

disabled opmode -> stop all moving mechanisms

Telemetry should show persistent subsystem state:

- drive mode

- intake state
- shooter target/current speed
- lift/arm target
- turret target/current angle
- sensor status
- whether automation is active

Emergency Stop Behavior For Mechanisms

FTC robots often have mechanisms powerful enough to damage themselves.

Every major mechanism should have an emergency stop or cancel behavior.

Mechanisms that need this:

- intake
- lift
- arm
- slide
- turret
- shooter
- transfer/indexer
- climber/hang

Emergency stop should:

- Stop motor power or switch to safe hold.
- Cancel the current command/state machine action.
- Leave servos in a safe position.
- Be easy for the operator to reach.
- Be tested on blocks before matches.

Recommended control pattern:

Gamepad 2 B or Back = cancel/stop mechanisms

For dangerous mechanisms, prefer hold-to-run:

```
while trigger held:
    move mechanism
else:
    stop or hold
```

For automated sequences, add a cancel path:

```
if cancel pressed:
    stop motors
    close gate
    return state to IDLE
```

TeleOp Testing Checklist

Before adding new complexity, test:

- Robot stops when both sticks are released.
- Slow mode works and does not change direction signs.
- Full-speed mode is controllable.
- Driver can strafe, rotate, and drive diagonally predictably.
- Driver and operator controls do not fight each other.
- Every running mechanism has an obvious stop.
- Emergency stop cancels active mechanism automation.
- Telemetry shows the important subsystem states.
- Controls are documented and match what the drive team practiced.
- Robot can be recovered from a bad state without restarting the OpMode.

Recommended First TeleOp Control Map

For a new team with mecanum drive and one simple mechanism:

Gamepad 1

left stick y -> forward/back
left stick x -> strafe
right stick x -> turn
right bumper -> slow mode
start/back -> reset heading, only if using field-centric

Gamepad 2

right trigger -> intake while held
left trigger -> reverse intake while held
A -> score/open servo while held or press once
B -> stop/cancel mechanism
dpad -> mechanism presets, if needed

Keep the first version boring and reliable. Add automation only after the manual controls are easy to use.

When To Add Complexity

Add more features only when the current TeleOp passes these checks:

- Drivers can explain every control without looking.
- Robot behaves safely when controls are released.
- Operators can recover from jams or bad positions.
- The same control map has survived multiple full practice matches.
- Telemetry makes failures understandable.

Good TeleOp is the foundation for everything else. Autonomous and advanced mechanisms are much easier when the robot already has clean controls, safe defaults, and drivers who trust the machine.